

JavaScript Essentials And Advanced

Theory based questions

1. JavaScript Introduction Theory Assignment

Question 1: What is JavaScript? Explain the role of JavaScript in web development.

Ans. JavaScript is a high-level, interpreted programming language that was originally designed to make web pages interactive. Alongside HTML (structure) and CSS (style), JavaScript is one of the three core technologies of the World Wide Web.

Role of JavaScript in web development:

1. **Adding Interactivity:** JavaScript allows developers to create dynamic user interfaces. Examples include dropdown menus, form validation (checking if an email is entered correctly before sending), image sliders, and showing/hiding content.
2. **Manipulating the DOM (Document Object Model):** JavaScript can read and modify the content, structure, and style of a web page in real-time without requiring the page to reload. For instance, changing the text of a paragraph or the color of a button when clicked.
3. **Client-Side Logic:** It handles logic directly in the user's browser, reducing the need to communicate with the server for every small action (e.g., showing an error message if a password is too short).

- 4. **Asynchronous Communication (AJAX/Fetch):** JavaScript can send and receive data from a web server in the background. This allows parts of a web page to update without refreshing the entire page (e.g., loading new tweets on Twitter or submitting a comment on a blog).
- 5. **Server-Side Development (with Node.js):** Modern JavaScript (using Node.js) can also run on web servers, allowing developers to use the same language for both the front-end (browser) and back-end (server).

Question 2: How is JavaScript different from other programming languages like Python or Java?

Ans. While JavaScript shares some basic syntax with Java and Python, it is fundamentally different in its design, execution, and primary use cases. The key differences are:

Feature	JavaScript	Python	Java
Primary Use	Web development (front-end & back-end), mobile apps	Data science, AI, scripting, back-end web	Enterprise software, Android apps, large systems
Execution	Interpreted (in browser or Node.js)	Interpreted	Compiled (to bytecode, then JVM)
Typing	Dynamically typed (variable type can change)	Dynamically typed	Statically typed (type is fixed)
Object Model	Prototype-based (objects inherit from other objects)	Class-based	Class-based

Concurrency	Single-threaded + Event Loop (asynchronous)	Multi-threaded (via threading library)	Multi-threaded (native threads)
Running Environment	Browser or Node.js	Python interpreter (local/ server)	Java Virtual Machine (JVM)
Syntax for Blocks	Curly braces {}	Indentation (whitespace)	Curly braces {}

Question 3: Discuss the use of tag in HTML. How can you link an external JavaScript file to an HTML document?

Ans. The `<script>` tag is used to embed or reference executable JavaScript code within an HTML document. Without this tag, the browser would treat JavaScript code as plain text and not execute it.

Uses of the `<script>` tag:

1. **Internal JavaScript:** Writing code directly between the opening `<script>` and closing `</script>` tags. This is useful for small, page-specific scripts.

```
<script>
```

```
    alert("Hello from internal script!");
```

```
</script>
```

1. **External JavaScript:** Linking to a separate .js file using the `src` attribute. This is the preferred method for larger projects because it promotes code reusability, keeps HTML clean, and allows browser caching.
2. **Controlling Loading Behavior:** Using attributes like `async` and `defer` to control when the script is fetched and executed, which improves page load performance.

How to Link an External JavaScript File

To link an external JavaScript file, you use the src (source) attribute inside the <script> tag. The src attribute contains the path to your .js file.

Syntax:

```
<script src="path/to/your-file.js"></script>
```

2. Variables and Data Types

Question 1: What are variables in JavaScript? How do you declare a variable using var, let, and const?

Ans. Variables are containers that store data values in JavaScript. They are declared using three keywords:

1. **var** (old way): Function-scoped, can be redeclared and reassigned. Not recommended in modern code.

```
var name = "John";
```

1. **let** (modern): Block-scoped, cannot be redeclared in same scope, but can be reassigned. Use when the value may change.

```
let age = 25;
```

```
age = 26; // Allowed
```

1. **const** (modern): Block-scoped, cannot be redeclared or reassigned. Must be initialized at declaration. Use by default.

```
const PI = 3.14;
```

```
// PI = 3.15; // Error
```

Question 2: Explain the different data types in JavaScript. Provide examples for each.

Ans. JavaScript has 8 main data types divided into two categories:

Primitive Data Types (7 types):

1. **String:** Text values. `let name = "Hello";`
2. **Number:** Integers and decimals. `let age = 25; let price = 19.99;`
3. **Boolean:** True or false. `let isActive = true;`
4. **Undefined:** Variable declared but no value assigned. `let x; // x is undefined`
5. **Null:** Intentionally empty value. `let y = null;`
6. **BigInt:** Very large numbers. `let big = 9007199254740991n;`
7. **Symbol:** Unique identifiers. `let sym = Symbol("id");`

Non-Primitive Data Type (1 type):

1. **Object:** Collections of key-value pairs, arrays, functions, and dates.

```
let person = { name: "John", age: 30 };
```

```
let colors = ["red", "green", "blue"];
```

Question 3: What is the difference between undefined and null in JavaScript?

Ans.

The main differences between undefined and null are:

Feature	undefined	null
Meaning	Variable is declared but has no value assigned	Intentionally empty or no value

Type	typeof undefined → "undefined"	typeof null → "object" (historical bug)
Assignment	Automatically assigned by JavaScript	Must be manually assigned by the developer
Purpose	Default value for uninitialized variables	Represents "nothing" or "empty" intentionally

3. JavaScript Operators Theory Assignment

Question 1: What are the different types of operators in JavaScript? Explain with examples.

Ans. Operators in JavaScript are symbols that perform operations on operands (values or variables). The main types are:

1. Arithmetic Operators – Perform mathematical calculations.

- **+ Addition** : $5 + 2 = 7$
- **- Subtraction** : $5 - 2 = 3$
- *** Multiplication** : $5 * 2 = 10$
- **/ Division** : $5 / 2 = 2.5$
- **% Modulus (remainder)** : $5 \% 2 = 1$
- **** Exponentiation** : $5 ** 2 = 25$

2. Assignment Operators – Assign values to variables.

- **= Assign** : `let x = 10;`

- **+= Add and assign** : `x += 5;` (same as `x = x + 5`)
- **-= Subtract and assign** : `x -= 3;`
- ***= Multiply and assign** : `x *= 2;`
- **/= Divide and assign**: `x /= 2;`

3. Comparison Operators – Compare values and return true or false.

- **== Equal to (value only)**: `5 == "5" → true`
- **=== Strict equal (value + type)**: `5 === "5" → false`
- **!= Not equal**: `5 != "5" → false`
- **!== Strict not equal**: `5 !== "5" → true`
- **> Greater than**: `10 > 5 → true`
- **< Less than**: `10 < 5 → false`
- **>= Greater than or equal**
- **<= Less than or equal**

4. Logical Operators – Combine boolean values/expressions.

- **&& AND: true && false** → false (both must be true)
- **|| OR: true || false** → true (at least one true)
- **! NOT: !true** → false (inverts the value)

Question 2: What is the difference between == and === in JavaScript?

Ans.The difference lies in **type coercion** (automatic type conversion).

Operator	Name	Behavior	Example	Result
<code>==</code>	Loose equality	Converts operands to same type before comparing	<code>5 == "5"</code>	true

===	Strict equality	Compares both value AND type without conversion	5 === "5"	false
------------	------------------------	---	-----------	-------

4. Control Flow (If-Else, Switch) Theory Assignment

Question 1: What is control flow in JavaScript? Explain how if-else statements work with an example.

Ans. Control Flow in JavaScript

Control flow means the order in which statements and instructions are executed in a JavaScript program. It helps the program make decisions and repeat tasks based on conditions.

JavaScript control flow statements include:

- if
- if-else
- switch
- loops (for, while)
- break, continue

if-else Statement

The if-else statement is used to execute different blocks of code depending on whether a condition is true or false.

Syntax

```
if(condition)
{
    // code if condition is true
}
else
{
    // code if condition is false
}
```

Example

```
let age = 18;

if(age >= 18)
{
    console.log("Eligible for voting");
}
```



```
else
{
    console.log("Not eligible for voting");
}
```

Question 2: Describe how switch statements work in JavaScript. When should you use a switch statement instead of if-else?

Ans. Switch Statement in JavaScript

The `switch` statement is used to select one block of code from multiple options. It compares a value with different `case` labels.

Syntax

```
switch(expression)
{
    case value1:
        // code
        break;

    case value2:
        // code
        break;

    default:
        // default code
}
```

Example

```
let day = 3;

switch(day)
{
    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
        break;

    case 3:
        console.log("Wednesday");
        break;

    default:
        console.log("Invalid day");
}
```

When to Use switch Instead of if-else

Use a `switch` statement when:

- There are many fixed values to compare.
- Multiple conditions depend on the same variable.
- Code readability and organization are important.

Use `if-else` when:

- Conditions involve ranges or logical operators (`>`, `<`, `&&`, `||`).
 - Complex expressions need to be checked.
-

5. Loops (For, While, Do-While) Theory Assignment

Question 1: Explain the different types of loops in JavaScript (for, while, do-while). Provide a basic example of each.

Ans. Loops in JavaScript

Loops are used to execute a block of code repeatedly until a condition becomes false.

The main types of loops in JavaScript are:

1. `for` loop
 2. `while` loop
 3. `do-while` loop
-

1. for Loop

Explanation

The `for` loop is used when the number of iterations is known in advance.

Syntax

```
for(initialization; condition; increment/decrement)
{
    // code
}
```

Example

```
for(let i = 1; i <= 5; i++)
{
    console.log(i);
}
```

Output

```
1
2
3
4
5
```

2. while Loop

Explanation

The `while` loop executes the code as long as the condition is true.

Syntax

```
while(condition)
{
    // code
}
```

Example

```
let i = 1;

while(i <= 5)
{
    console.log(i);
    i++;
}
```

Output

```
1
2
3
4
5
```

3. do-while Loop

Explanation

The `do-while` loop executes the block of code first and then checks the condition.

Syntax

```
do
{
    // code
}
while(condition);
```

Example

```
let i = 1;

do
{
    console.log(i);
    i++;
}
while(i <= 5);
```

Output

```
1
2
3
4
5
```

Question 2: What is the difference between a while loop and a do-while loop?

Ans.

while Loop

Condition is checked before executing the loop body.

May execute zero times if condition is false initially.

Syntax starts with `while`.

do-while Loop

Condition is checked after executing the loop body.

Executes at least one time even if condition is false.

Syntax starts with `do`.

Example of while Loop

```
let x = 10;

while(x < 5)
{
    console.log(x);
}
```

Output: No output

Example of do-while Loop

```
let x = 10;

do
{
    console.log(x);
}
while(x < 5);
```

Output:

6. Functions Theory Assignment

Question 1: What are functions in JavaScript? Explain the syntax for declaring and calling a function.

Ans. Functions in JavaScript

A function is a block of code designed to perform a specific task. Functions help in code reusability, organization, and reducing repetition.

A function executes only when it is called.

Syntax for Declaring a Function

```
function functionName(parameters)
{
    // code to execute
}
```

Example of Function Declaration

```
function greet()
{
    console.log("Hello World");
}
```

Calling a Function

To execute a function, its name is written followed by parentheses ().

```
greet();
```

Complete Example

```
function greet()
{
    console.log("Hello World");
}

greet();
```

Output

Hello World

Question 2: What is the difference between a function declaration and a function expression?

Ans.

Function Declaration

Defined using the `function` keyword with a function name.

Can be called before declaration due to hoisting.

Syntax is simple and commonly used.

Function Expression

Function is stored inside a variable.

Cannot be called before declaration.

Useful for anonymous functions and callbacks.

Function Declaration Example

```
function add(a, b)
{
    return a + b;
}

console.log(add(2, 3));
```

Output

5

Function Expression Example

```
const add = function(a, b)
{
    return a + b;
};

console.log(add(2, 3));
```

Output

5

Question 3: Discuss the concept of parameters and return values in functions.

Ans. Parameters

Parameters are variables listed in the function definition. They receive values when the function is called.

Example

```
function greet(name)
{
    console.log("Hello " + name);
}

greet("Ayush");
```

Explanation

- `name` is a parameter.
 - `"Ayush"` is an argument passed to the function.
-

Return Values

A return value is the value that a function sends back using the `return` statement.

Example

```
function multiply(a, b)
{
    return a * b;
}

let result = multiply(4, 5);

console.log(result);
```

Output

20

7. Arrays Theory Assignment

Question 1: What is an array in JavaScript? How do you declare and initialize an array?

Ans. An array in JavaScript is a special variable used to store multiple values in a single variable. Arrays can store numbers, strings, objects, and other data types.

Syntax

```
let arrayName = [value1, value2, value3];
```

Example

```
let fruits = ["Apple", "Banana", "Mango"];
```

In this example:

- `fruits` is the array name.
- `"Apple"`, `"Banana"`, and `"Mango"` are array elements.

Array elements are accessed using index numbers starting from 0.

```
console.log(fruits[0]);
```

Output

Apple

Question 2: Explain the methods `push()`, `pop()`, `shift()`, and `unshift()` used in arrays.

Ans. 1. `push()`

The `push()` method adds one or more elements to the end of an array.

```
let arr = [1, 2];  
arr.push(3);  
  
console.log(arr);
```

Output

[1, 2, 3]

2. `pop()`

The `pop()` method removes the last element from an array.

```
let arr = [1, 2, 3];  
arr.pop();  
  
console.log(arr);
```

Output


```
[1, 2]
```

3. shift()

The `shift()` method removes the first element from an array.

```
let arr = [1, 2, 3];
arr.shift();

console.log(arr);
```

Output

```
[2, 3]
```

4. unshift()

The `unshift()` method adds one or more elements at the beginning of an array.

```
let arr = [2, 3];
arr.unshift(1);

console.log(arr);
```

Output

```
[1, 2, 3]
```

8. Objects Theory Assignment

Question 1: What is an object in JavaScript? How are objects different from arrays?

Ans. An object in JavaScript is a collection of properties stored as key-value pairs. Objects are used to represent real-world entities and store related data.

Example

```
let student = {
  name: "Ayush",
  age: 20,
  city: "Ahmedabad"
};
```

Difference Between Objects and Arrays

Objects	Arrays
Store data in key-value pairs	Store ordered values
Accessed using property names	Accessed using index numbers
Used for representing entities	Used for storing lists of data

Question 2: Explain how to access and update object properties using dot notation and bracket notation.

Ans. Dot Notation

Properties are accessed using a dot (.).

Example

```
let student = {  
  name: "Ayush",  
  age: 20  
};  
  
console.log(student.name);
```

Output

Ayush

Updating Property

```
student.age = 21;  
  
console.log(student.age);
```

Output

21

Bracket Notation

Properties are accessed using square brackets [].

Example

```
let student = {  
  name: "Ayush",  
  age: 20  
};  
  
console.log(student["name"]);
```

Output

Ayush

Updating Property

```
student["age"] = 22;  
  
console.log(student["age"]);
```

Output

22

9. JavaScript Events Theory Assignment

Question 1: What are JavaScript events? Explain the role of event listeners.

Ans. JavaScript events are actions or occurrences that happen in the browser, such as clicking a button, pressing a key, or moving the mouse.

Examples of events:

- click
- mouseover
- keydown
- submit

Event listeners are functions that wait for an event to occur and then execute specific code.

Example

```
button.addEventListener("click", function() {  
  console.log("Button clicked");  
});
```

In this example, the event listener waits for the button click event and runs the function when the event occurs.

Question 2: How does the `addEventListener()` method work in JavaScript?

Provide an example.

Ans. The `addEventListener()` method attaches an event handler to an HTML element without overwriting existing events.

Syntax

```
element.addEventListener(event, function);
```

Example

```
let btn = document.getElementById("myBtn");

btn.addEventListener("click", function() {
    alert("Button Clicked");
});
```

10. DOM Manipulation Theory Assignment

Question 1: What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?

Ans. The DOM (Document Object Model) is a programming interface for HTML documents. It represents the webpage as a tree structure of objects.

JavaScript interacts with the DOM to:

- Access HTML elements
- Change content and styles
- Add or remove elements
- Handle events

Example

```
document.getElementById("demo").innerHTML = "Hello";
```

This changes the content of the HTML element with id "demo".

Question 2: Explain the methods `getElementById()`, `getElementsByClassName()`, and `querySelector()` used to select elements from the DOM.

Ans. 1. `getElementById()`

Selects an element using its id.

Example

```
let element = document.getElementById("demo");
```

2. `getElementsByClassName()`

Selects all elements with a specific class name.

Example

```
let items = document.getElementsByClassName("box");
```

3. `querySelector()`

Selects the first element that matches a CSS selector.

Example

```
let element = document.querySelector(".box");
```

Difference

- `getElementById()` selects by id.
 - `getElementsByClassName()` selects multiple elements by class.
 - `querySelector()` uses CSS selectors.
-

11. JavaScript Timing Events (`setTimeout`, `setInterval`)

Question 1: Explain the `setTimeout()` and `setInterval()` functions in JavaScript. How are they used for timing events?

Ans. setTimeout()

The `setTimeout()` function executes code once after a specified time delay.

Syntax

```
setTimeout(function, time);
```

setInterval()

The `setInterval()` function repeatedly executes code at specified intervals.

Syntax

```
setInterval(function, time);
```

Example

```
setInterval(function() {  
    console.log("Hello");  
}, 1000);
```

This prints "Hello" every 1 second.

These functions are used for:

- Delaying actions
- Creating timers
- Animations
- Automatic updates

Question 2: Provide an example of how to use `setTimeout()` to delay an action by 2 seconds.

Ans.

```
setTimeout(function() {  
    console.log("Action executed after 2 seconds");  
}, 2000);
```

Explanation

- 2000 milliseconds = 2 seconds.
- The function executes after the delay.

12. JavaScript Error Handling Theory Assignment

Question 1: What is error handling in JavaScript? Explain the try, catch, and finally blocks with an example.

Ans. Error handling is the process of managing runtime errors in a program so that the application does not crash unexpectedly.

JavaScript uses:

- try
- catch
- finally

blocks for error handling.

Example

```
try
{
    let result = 10 / 0;
    console.log(result);
}
catch(error)
{
    console.log("An error occurred");
}
finally
{
    console.log("Execution completed");
}
```

Explanation

- try contains code that may produce an error.
- catch handles the error.
- finally always executes whether an error occurs or not.

Question 2: Why is error handling important in JavaScript applications?

Ans. Error handling is important because it:

- Prevents application crashes
- Improves user experience

- Helps identify and debug problems
- Maintains smooth execution of programs
- Makes applications more reliable and secure

=====

Lab based tasks

1. Task:

Create a simple HTML page and add a tag within the page.

Write JavaScript code to display an alert box with the message "Welcome to JavaScript!" when the page loads.<!--EndFragment--> </body> </html>

Ans. <!DOCTYPE html>

<html>

<head>

<title>JavaScript Alert Example</title>

</head>

<body>

<h1>Welcome Page</h1>

<script>

alert("Welcome to JavaScript!");

</script>

</body>

</html>

2. Task:

Write a JavaScript program to declare variables for different data types (string, number, boolean, null, and undefined).

Ans. // String


```
let name = "Ayush";  
  
// Number  
  
let age = 20;  
  
// Boolean  
  
let isStudent = true;  
  
// Null  
  
let data = null;  
  
// Undefined  
  
let city;
```

Log the values of the variables and their types to the console using `console.log()`.

Ans. // Display values

```
console.log(name);  
  
console.log(age);  
  
console.log(isStudent);  
  
console.log(data);  
  
console.log(city);
```

// Display types

```
console.log(typeof name);  
  
console.log(typeof age);  
  
console.log(typeof isStudent);  
  
console.log(typeof data);  
  
console.log(typeof city);
```

3. Task:

Create a JavaScript program to perform the following:

- Add, subtract, multiply, and divide two numbers using arithmetic operators.
- Use comparison operators to check if two numbers are equal and if one number is greater than the other.
- Use logical operators to check if both conditions (e.g., $a > 10$ and $b < 5$) are true.

Ans. // Declare two numbers

```
let a = 20;
```

```
let b = 5;
```

```
// Arithmetic Operators
```

```
console.log("Addition: " + (a + b));
```

```
console.log("Subtraction: " + (a - b));
```

```
console.log("Multiplication: " + (a * b));
```

```
console.log("Division: " + (a / b));
```

```
// Comparison Operators
```

```
console.log("Are numbers equal? " + (a == b));
```

```
console.log("Is a greater than b? " + (a > b));
```

```
// Logical Operators
```

```
console.log("Both conditions are true: " + (a > 10 && b < 5));
```

Output

```
Addition: 25
Subtraction: 15
Multiplication: 100
Division: 4
Are numbers equal? false
Is a greater than b? true
Both conditions are true: false
```

4. Task:

1: Write a JavaScript program to check if a number is positive, negative, or zero using an if-else statement.

Ans. `let number = -5;`

```
if(number > 0)
{
    console.log("Positive Number");
}
else if(number < 0)
{
    console.log("Negative Number");
}
else
{
    console.log("Zero");
}
```

Output

Negative Number

2: Create a JavaScript program using a switch statement to display the day of the week based on the user input (e.g., 1 for Monday, 2 for Tuesday, etc.).

Ans. `let day = 3;`

```
switch(day)
{
    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
        break;

    case 3:
        console.log("Wednesday");
        break;

    case 4:
        console.log("Thursday");
        break;

    case 5:
        console.log("Friday");
        break;

    case 6:
```

```
        console.log("Saturday");
        break;

    case 7:
        console.log("Sunday");
        break;

    default:
        console.log("Invalid Input");
}

```

Output

Wednesday

5. Task:

1: Write a JavaScript program using a for loop to print numbers from 1 to 10.

```
for(let i = 1; i <= 10; i++)
{
    console.log(i);
}

```

Output

1
2
3
4
5
6
7
8
9
10

2: Create a JavaScript program that uses a while loop to sum all even numbers between 1 and 20.

```
let i = 2;
let sum = 0;

while(i <= 20)
{
    sum = sum + i;
    i = i + 2;
}

console.log("Sum of even numbers: " + sum);

```

Output

```
Sum of even numbers: 110
```

3: Write a do-while loop that continues to ask the user for input until they enter a number greater than 10.

```
let number;
```

```
do
```

```
{
```

```
  number = parseInt(prompt("Enter a number greater than 10:"));
```

```
}
```

```
while(number <= 10);
```

```
console.log("You entered: " + number);
```

6. Task:

1: Write a function greetUser that accepts a user's name as a parameter and displays a greeting message (e.g., "Hello, John!").

```
function greetUser(name)
{
  console.log("Hello, " + name + "!");
}

greetUser("John");
```

Output

```
Hello, John!
```

2: Create a JavaScript function calculateSum that takes two numbers as parameters, adds them, and returns the result

```
function calculateSum(a, b)
{
  return a + b;
}

let result = calculateSum(10, 20);
```

```
console.log("Sum = " + result);
```

Output

```
Sum = 30
```

7. Task:

1: Declare an array of fruits (["apple", "banana", "cherry"]). Use JavaScript to:

- **Add a fruit to the end of the array.**
- **Remove the first fruit from the array.**
- **Log the modified array to the console.**

Ans.

```
let fruits = ["apple", "banana", "cherry"];
```

```
// Add a fruit to the end  
fruits.push("mango");
```

```
// Remove the first fruit  
fruits.shift();
```

```
// Display modified array  
console.log(fruits);
```

Output

```
["banana", "cherry", "mango"]
```

2: Write a program to find the sum of all elements in an array of numbers.

```
let numbers = [10, 20, 30, 40];  
let sum = 0;  
  
for(let i = 0; i < numbers.length; i++)  
{  
    sum = sum + numbers[i];  
}  
  
console.log("Sum = " + sum);
```

Output

```
Sum = 100
```

8. Task:

Create a JavaScript object car with properties brand, model, and year.

Use JavaScript to:

- Access and print the car's brand and model.
- Update the year property.
- Add a new property color to the car object.

```
let car = {  
  brand: "Toyota",  
  model: "Fortuner",  
  year: 2022  
};  
  
// Access and print brand and model  
console.log(car.brand);  
console.log(car.model);  
  
// Update year property  
car.year = 2024;  
  
// Add new property  
car.color = "Black";  
  
// Display complete object  
console.log(car);
```

Output

```
Toyota  
Fortuner  
{  
  brand: "Toyota",  
  model: "Fortuner",  
  year: 2024,  
  color: "Black"  
}
```

9. Task:

Create a simple webpage with a button that, when clicked, displays an alert saying "Button clicked!" using JavaScript event listeners.

```
<!DOCTYPE html>
<html>
<head>
<title>Button Click Event</title>
</head>

<body>

<button id="btn">Click Me</button>

<script>
let button = document.getElementById("btn");

button.addEventListener("click", function() {
alert("Button clicked!");
});
</script>

</body>
</html>
```

10. Task:

Create an HTML page with a paragraph () that displays "Hello, World!".

Use JavaScript to:

- Change the text inside the paragraph to "JavaScript is fun!".
- Change the color of the paragraph to blue.


```
<html>
<head>
<title>DOM Manipulation</title>
</head>

<body>

<p id="demo">Hello, World!</p>

<script>
let para = document.getElementById("demo");

// Change text
para.innerHTML = "JavaScript is fun!";

// Change color
para.style.color = "blue";
</script>

</body>
</html>
```

11. Task:

1: Write a program that changes the background color of a webpage after 5 seconds using setTimeout().

2: Create a digital clock that updates every second using setInterval().

```
<!DOCTYPE html>
<html>
<head>
  <title>Background Color Change</title>
</head>

<body>

  <h1>Background color will change after 5 seconds</h1>
```

```
<script>
    setTimeout(function() {
        document.body.style.backgroundColor = "lightblue";
    }, 5000);
</script>

</body>
</html>
```

12. Task:

Write a JavaScript program that attempts to divide a number by zero. Use `trycatch` to handle the error and display an appropriate error message.

```
<body>

<h1>Digital Clock</h1>

<h2 id="clock"></h2>

<script>
    setInterval(function() {

        let now = new Date();

        let time = now.toLocaleTimeString();

        document.getElementById("clock").innerHTML = time;

    }, 1000);
</script>

</body>
</html>
```